PetAdopt+ — Viva Cheat Sheet (1–2 pages)

Project: Pet care platform — Adoption + Online Shop + Pet Hostel

Tech: React (port 5173) | Node/Express (port 4000) | MongoDB Atlas | Cloudinary | Khalti | Gmail

---

30-Second Elevator Pitch

> "PetAdopt+ is a web app for a pet shelter. Users can **adopt pets**, **buy pet products** online, and **book hostel rooms** for their pets. Admins manage everything from a dashboard with revenue charts. Staff handle hostel check-ins. Images use **Cloudinary**, payments use **Khalti**, and data is stored in **MongoDB**."

Start Demo (memorize)

```bash

# Terminal 1

cd backend && npm run dev

# Wait: "Server running on port 4000" + "Connected to MongoDB"

# Terminal 2

cd frontend && npm run dev

# Open: http://localhost:5173

```

Stuck? cd backend && npm run kill-port && npm run dev

Architecture (draw on board if asked)

```
Browser (React) → API (Express) → MongoDB
↓
Cloudinary | Khalti | Email
```

- **Frontend:** pages, cart (browser storage), sends JSON + token
- **Backend:** Routes → Controllers → Models
- **Token:** saved in localStorage after login; sent as Authorization: Bearer ...

Three User Roles

Role	After login goes to	Main job
user	Shop / browse pets	Buy, adopt, book hostel
admin	Admin dashboard	Manage all + charts
staff	Staff dashboard	Rooms & bookings only

Main Features (one line each)

Feature	What happens
Signup	Register → 6-digit email code → verify → login
Adoption	Form → status pending → admin approve/reject → email
Shop	Products → cart → checkout → COD or Khalti
Hostel	Pick room + dates → check availability → book → pay
Dashboard	Stats cards + **bar chart** (monthly revenue) + **pie chart** (top products)
Upload	Image → server → **Cloudinary** → URL saved in database

---

Cloudinary (common viva question)

1. `Admin picks image →` POST /api/upload
2. `File saved briefly in` backend/uploads/
3. `Uploaded to **Cloudinary** folder` petadopt
4. `Returns **URL** → saved in pet/product/room` image `field`
5. Website shows image from that URL (not from MongoDB file storage)

Why? MongoDB is for data, not big files. Cloudinary handles storage + fast delivery.

Discount (common viva question)

- `Stored on product:` price `+` discount `(0–100%)`

- **Formula:** Offer price = price × (1 - discount/100)
- Example: Rs 1000, 20% off → **Rs 800**
- Shop shows crossed-out price + badge "20% OFF"
- Cart uses discounted price when user adds to cart

Dashboard Bar Graph (common viva question)

Step	Detail
API	GET /api/dashboard/revenue
Counts	Delivered + **Paid** orders + hostel bookings (not cancelled)
Shows	12 bars (Jan–Dec), height = orders + bookings revenue
Click bar	GET /api/orders/revenue/:month/:year → list of orders & bookings
Pie chart	GET /api/orders/product-sales → top products still in inventory

---

Authentication (common viva question)

1. Login → backend checks password (encrypted with bcrypt)
2. Must be **email verified**
3. Returns **JWT token** (expires in 7 days)
4. Protected pages: frontend ProtectedRoute + backend protect middleware
5. Admin routes: extra adminOnly check

Key API Prefixes

Prefix	Purpose
`/api/auth`	Login, signup, profile
`/api/pets`	Adoption animals
`/api/products`	Shop inventory
`/api/orders`	Shop orders + sales charts
`/api/hostel-rooms`	Rooms
`/api/hostel-bookings`	Bookings
`/api/adoptions`	Adoption applications
`/api/dashboard`	Admin stats & revenue
`/api/upload`	Images
`/api/payments`	Khalti

Database Collections (9)

`User` | `Pet` | `Product` | `Order` | `AdoptionApplication` | `HostelRoom` | `HostelBooking` | `ProductReview` | `HostelRoomReview`

Order example fields: `orderNumber, items[], subtotal, shipping, total, status, paymentStatus`

Booking statuses: `Pending → Confirmed → Checked-In → Checked-Out`

Free shipping: `subtotal ≥ Rs 10,000`

Error Handling (if asked)

- `Backend:` try/catch `→ JSON` { success: false, message: "..." }
- **401** = not logged in | **403** = wrong role | **404** = not found | **400** = validation

- `Frontend: checks success, shows alert or error text`

Demo Flow (5 minutes — practice this)

1. **Landing page** → explain three services
2. **Login as user** → shop → add to cart → show discount if any
3. **Login as admin** → dashboard → point at **bar chart** → click one month
4. **Admin** → pets or inventory → mention **Cloudinary** on image upload
5. **Optional:** hostel booking or adoption status

Likely Questions & Short Answers

Q: Why React + Node separate?

A: Frontend for UI, backend for security and database. API connects them.

Q: Why MongoDB?

A: Flexible documents (orders with nested items, bookings with pet details).

Q: How is password stored?

A: Hashed with bcrypt — never stored as plain text.

Q: What if Cloudinary is down?

A: Upload fails; rest of app works. Images need Cloudinary or another URL.

Q: COD vs Khalti?

A: COD confirms order immediately and reduces stock. Khalti pays online first, then verify reduces stock.

Q: Who can see admin dashboard?

`A: Only users with role: admin and valid token.`

Files to Name if Examiner Asks “Show me code”

Topic	File
Server start	backend/index.js
Login	backend/Controller/AuthController.js
Auth check	backend/Middleware/Auth.js
Cloudinary	backend/config/cloudinary.js , UploadController.js
Revenue chart API	backend/Controller/DashboardController.js
Bar chart UI	frontend/src/pages/AdminDashboard.jsx
Discount display	frontend/src/pages/Shop.jsx
Routes	frontend/src/App.jsx

Before You Enter the Room

- [ ] Both terminals running, browser tab open on homepage
- [ ] Know admin + user test emails/passwords
- [ ] Internet on (MongoDB Atlas + Khalti need network)

- [ ] backend/.env configured (don’t show secrets on screen)

- [ ] Full doc: SYSTEM_DOCUMENTATION.md` on laptop if deep questions come

Good luck on Sunday.